

온톨로지 부트캠프 - 2일차

온톨로지와 기술요소

정 용 일 | kevin.jeong@saltlux.com



1. 기본개념

도메인 지식을 구조화하여 공유하고, 분산된 데이터를 공통 의미로 통합

도메인 지식 표현 / 공유

도메인 전문가의 지식을 구조화하여 모두가 이해할 수 있는 공통 의미로 만듭니다.

도메인 영역(현실 세계)



제품 · 공정 · 설비 · 사람 · 조직 · 공급업체 등
실제 비즈니스 환경

제품은 어떤
자재로 만들지요?



도메인 전문가
업무 지식

설비와 공정의
관계를 정리해봅시다.



데이터 분석가
요구사항 정리

이 개념을
데이터와 연결해요.



개발자
시스템 적용



개념 정의
개념의 의미와
범위 기술



관계 정의
개념 간 관계
(예: 생산, 사용)



공통 용어
도메인 용어의
표준화



지식 공유
모두가 이해하는
공통 모델

개념화 (개념 · 객체 · 관계)

도메인 지식을 핵심 개념과 관계로 추상화하여 개념모델을 만듭니다.



개념화

공통 의미 부여
연계 · 통합

온톨로지 레이어

개념모델 · 관계 · 공통의미

통합된 의미로 데이터와 지식을 연결

명세 / 스키마

개념모델을 형식화하여 클래스, 관계, 속성, 제약으로 명세합니다.



클래스

Product
Material
Machine
Employee
...



관계

생산한다
공급한다
소속된다
사용한다
...



속성

제품명
설비ID
담당자명
부서명
...



제약

제품은
하나 이상의
자재를
사용함
...

도메인 내 데이터 통합

분산된 데이터를 온톨로지의 공통 의미로 연결하여
하나의 도메인 데이터로 통합합니다.

HR 시스템

emp_id	name	dept_id
E001	홍길동	D001
E002	김철수	D002

ERP 시스템

product_id	material_id	supplier_id
P001	M001	S001
P002	M002	S002

MES 시스템

lot_id	product_id	machine_id
L001	P001	MC01
L002	P002	MC02

QMS 시스템

lot_id	result	inspect_date
L001	PASS	2024-09-01
L002	FAIL	2024-09-02

통합 조회 / 분석

지식과 데이터가 연결된 통합된 관점에서
다양한 인사이트를 얻을 수 있습니다.

통합 데이터 예시

제품	자재	공정	설비	담당자
P001	M001	조립공정	MC01	홍길동
P002	M002	검사공정	MC02	김철수
...				



관계 분석
개념 간 연관성 탐색



통합 조회
도메인 데이터 통합 검색



시각화
지식과 데이터 기반 인사이트



지식표현언어 Description Logics

Syntax for DLs

- Description logics: SHOIN(D), Terminological Logics, DLs
- Fragments of FOL
- W3C Standard OWL DL corresponds to SHOIN(D)
- DL 오퍼레이터와 명명규칙

$\mathcal{F}$ Functional properties.

$\mathcal{E}$ Full existential qualification (Existential restrictions that have fillers other than owl:thing).

$\mathcal{U}$ Concept union.

$\mathcal{C}$ Complex concept negation.

$\mathcal{S}$ An abbreviation for $\mathcal{ALL}$ with transitive roles.

$\mathcal{H}$ Role hierarchy (subproperties - rdfs:subPropertyOf).

$\mathcal{R}$ Limited complex role inclusion axioms; reflexivity and irreflexivity; role disjointness.

$\mathcal{O}$ Nominals. (Enumerated classes of object value restrictions - owl:oneOf, owl:hasValue).

$\mathcal{I}$ Inverse properties.

$\mathcal{N}$ Cardinality restrictions (owl:Cardinality, owl:MaxCardinality).

$\mathcal{Q}$ Qualified cardinality restrictions (available in OWL 1.1, cardinality restrictions that have fillers other than owl:thing).

(D) Use of datatype properties, data values or data types.

- $\mathcal{H}$ for role inclusion axioms (role hierarchy)
- $\mathcal{O}$ for nominals
- $\mathcal{I}$ for inverse roles
- $\mathcal{N}$ for number restrictions (of form $\leq nR, \geq nR$)
- $\mathcal{Q}$ for qualified number restrictions (of form $\leq nR.C, \geq nR.C$)

Concepts		
ALC	Atomic	A, B
	Not	$\neg C$
	And	$C \sqcap D$
	Or	$C \sqcup D$
	Exists	$\exists R.C$
QNI	For all	$\forall R.C$
	At least	$\geq n R.C$ ($\geq n R$)
	At most	$\leq n R.C$ ($\leq n R$)
O	Nominal	$\{i_1, \dots, i_n\}$

Roles		
-	Atomic	R
	Inverse	R^-

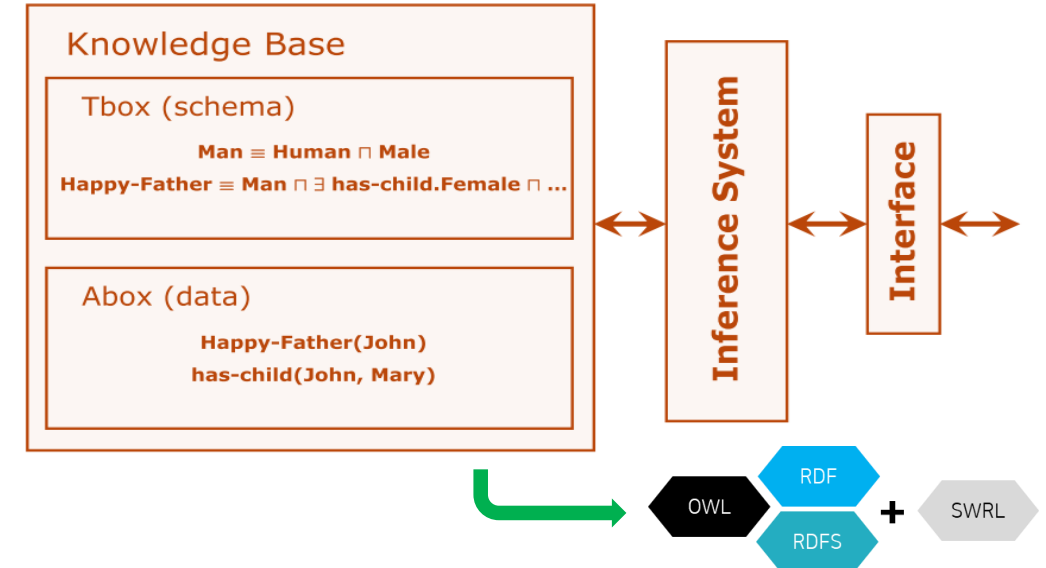
S = ALC + Transitivity

Ontology (=Knowledge Base)		
Concept Axioms (TBox)		
Subclass	$C \sqsubseteq D$	
Equivalent	$C \equiv D$	
Role Axioms (RBox)		
Subrole	$R \sqsubseteq S$	
Transitivity	Trans (S)	
Assertional Axioms (ABox)		
Instance	$C(a)$	
Role	$R(a, b)$	
Same	$a = b$	
Different	$a \neq b$	

OWL DL = SHOIN(D) (D: concrete domain)



General DL Architecture



- DL knowledge bases consist of two parts(in general)
TBox: Axioms, describing the structure of a modelled domain (conceptual schema):
 - $\text{HappyFather} \equiv \text{Man} \sqcap \exists \text{hasChild.Female} \sqcap \dots$
 - $\text{Elephant} \sqsubseteq \text{Animal} \sqcap \text{Large} \sqcap \text{Grey}$
 - $\text{transitive}(\text{hasAncestor})$

Abox: Axiome describing concrete situations (data, facts):

- HappyFather(John)
- $\text{hasChild(John, Mary)}$

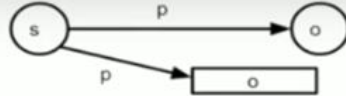
다운로드: [axioms_example.zip](#)



지식표현언어: RDF와 LPG 특징(1/2)

RDF

(s p o)



Vertices

Resources : URIs

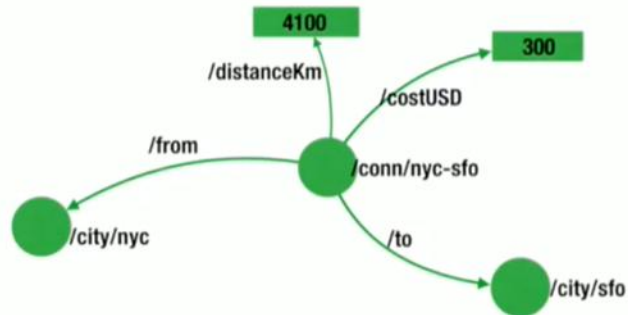
Attribute Values : Literal Values

Edges

Relationships : URIs

Nodes or Edges have NO internal structure

RDF



LPG



Vertices

Nodes : ID + set of key-value pairs

Edges

Relationships : ID + Type + set of key-value pairs

Nodes and Edges have internal structure

LPG (Neo4j)





지식표현언어: RDF와 LPG 특징(2/2)

Cypher	SPARQL
CREATE	INSERT
RETURN	SELECT
WITH	SELECT in a sub query
MATCH	WHERE
WHERE	FILTER
:label	owl:Class or rdfs:Class
[edge]	Predicate <edge> or Triple << <subject> <edge> <object> >>
(node)	a graph node (blank or IRI node)
var:Person	?var a :Person
var.name	?name assuming a match ?var <name> ?name
nodeVar:label {key value, key2 value2}	?nodeVar a label; <key> value;key2 value2.
(ee)-[:KNOWS {since: 2001}]->(js)	<< ?ee <knows> <js> >> <since> 2001.

SPARQL

```
SELECT ?ee ?friends
WHERE {?ee a <Person>;
       <name> "Emil";
       <knows> ?friends }
```

Cypher

```
MATCH (ee:Person)-[:KNOWS]-(friends)
WHERE ee.name = "Emil"
RETURN ee, friends
```

Cypher

```
CREATE (TheMatrix:Movie {title:'The Matrix', released:1999,
tagline:'Welcome to the Real World'})
CREATE (Keanu:Person {name:'Keanu Reeves', born:1964})
CREATE (Carrie:Person {name:'Carrie-Anne Moss', born:1967})
CREATE (Laurence:Person {name:'Laurence Fishburne', born:1961})
CREATE (Hugo:Person {name:'Hugo Weaving', born:1960})
CREATE (AndyW:Person {name:'Andy Wachowski', born:1967})
CREATE (LanaW:Person {name:'Lana Wachowski', born:1965})
CREATE (JoelS:Person {name:'Joel Silver', born:1952})
CREATE (Keanu)-[:ACTED_IN {roles:['Neo']}]->(TheMatrix),
(Carrie)-[:ACTED_IN {roles:['Trinity']}]->(TheMatrix),
(Laurence)-[:ACTED_IN {roles:['Morpheus']}]->(TheMatrix),
(Hugo)-[:ACTED_IN {roles:['Agent Smith']}]->(TheMatrix),
(AndyW)-[:DIRECTED]->(TheMatrix),
(LanaW)-[:DIRECTED]->(TheMatrix),
(JoelS)-[:PRODUCED]->(TheMatrix)
```

SPARQL

```
INSERT DATA {<TheMatrix> a <Movie>; <title>'The Matrix';
<released> 1999; <tagline>'Welcome to the Real World'.
<Keanu> a <Person>; <name>'Keanu Reeves'; <born> 1964.
<Carrie> a <Person>; <name>'Carrie-Anne Moss'; <born> 1967.
<Laurence> a <Person>; <name>'Laurence Fishburne'; <born> 1961.
<Hugo> a <Person>; <name>'Hugo Weaving'; <born> 1960.
<AndyW> a <Person>; <name>'Andy Wachowski'; <born> 1967.
<LanaW> a <Person>; <name>'Lana Wachowski'; <born> 1965.
<JoelS> a <Person> <name>'Joel Silver'; <born> 1952.
<< <Keanu>-<ACTED_IN> <TheMatrix> >> <roles> 'Neo'.
<< <Carrie>-<ACTED_IN> <TheMatrix> >> <roles> 'Trinity'.
<< <Laurence> <ACTED_IN> <TheMatrix> >> <roles> 'Morpheus'.
<< <Hugo> <ACTED_IN> <TheMatrix> >> <roles> 'Agent Smith'.
<AndyW> <DIRECTED> <TheMatrix>.
<LanaW> <DIRECTED> <TheMatrix>.
<JoelS> <PRODUCED><TheMatrix>.}
```

Cypher

```
MATCH (ee:Person)
WHERE ee.name = "Emil"
CREATE
(js:Person { name: "Johan", from: "Sweden",
learn: "surfing" }),
(ir:Person { name: "Ian", from: "England",
title: "author" }),
(rvb:Person { name: "Rik", from: "Belgium",
pet: "Orval" }),
(ally:Person { name: "Allison", from: "California",
hobby: "surfing" }),
(ee)-[:KNOWS {since: 2001}]->(js),
(ee)-[:KNOWS {rating: 5}]->(ir),
(js)-[:KNOWS]->(ir),(js)-[:KNOWS]->(rvb),
(ir)-[:KNOWS]->(js),(ir)-[:KNOWS]->(ally),
(rvb)-[:KNOWS]->(ally)
```

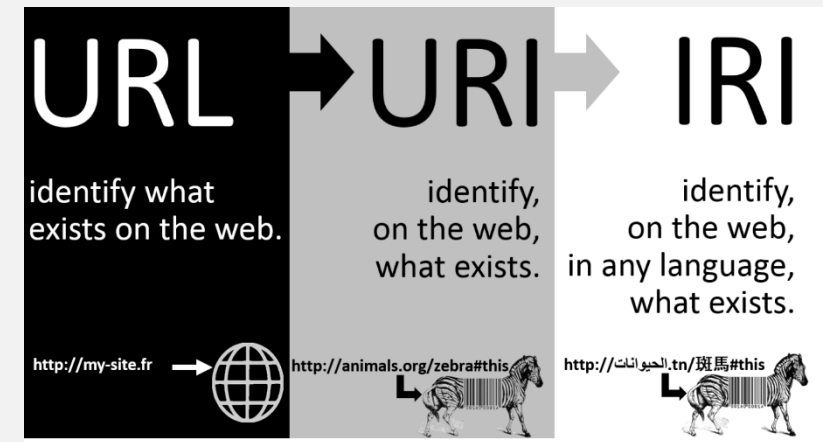
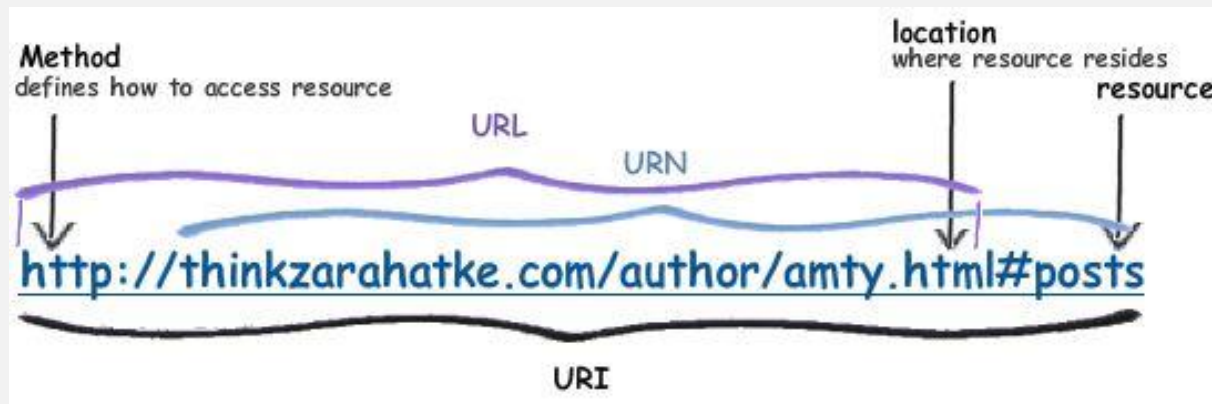
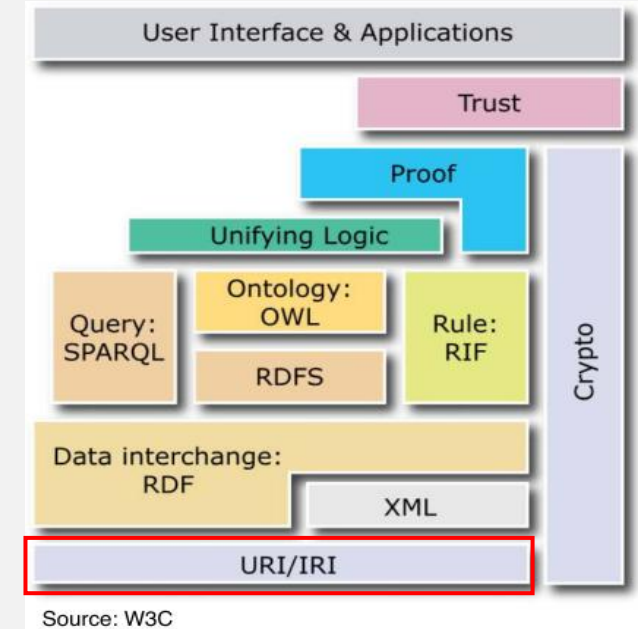
SPARQL

```
INSERT {
<js> a <Person>; <name> "Johan"; <from> "Sweden";
<learn> "surfing".
<ir> a <Person>; <name> "Ian"; <from> "England";
<title> "author".
<rvb> a <Person>; <name> "Rik"; <from> "Belgium";
<pet> "Orval".
<ally> a <Person>; <name> "Allison";
<from> "California"; <hobby> "surfing".
<<?ee <knows> <js>>> <since> "2001"^^xsd:string.
<<?ee <knows> <ir>>> <rating> "5"^^xsd:Integer.
<js> <knows> <ir>.
<js> <knows> <rvb>.
<ir> <knows> <js>.
<ir> <knows> <ally>.
<rvb> <knows> <ally>}
WHERE {?ee a <Person>; <name> "Emil"}
```



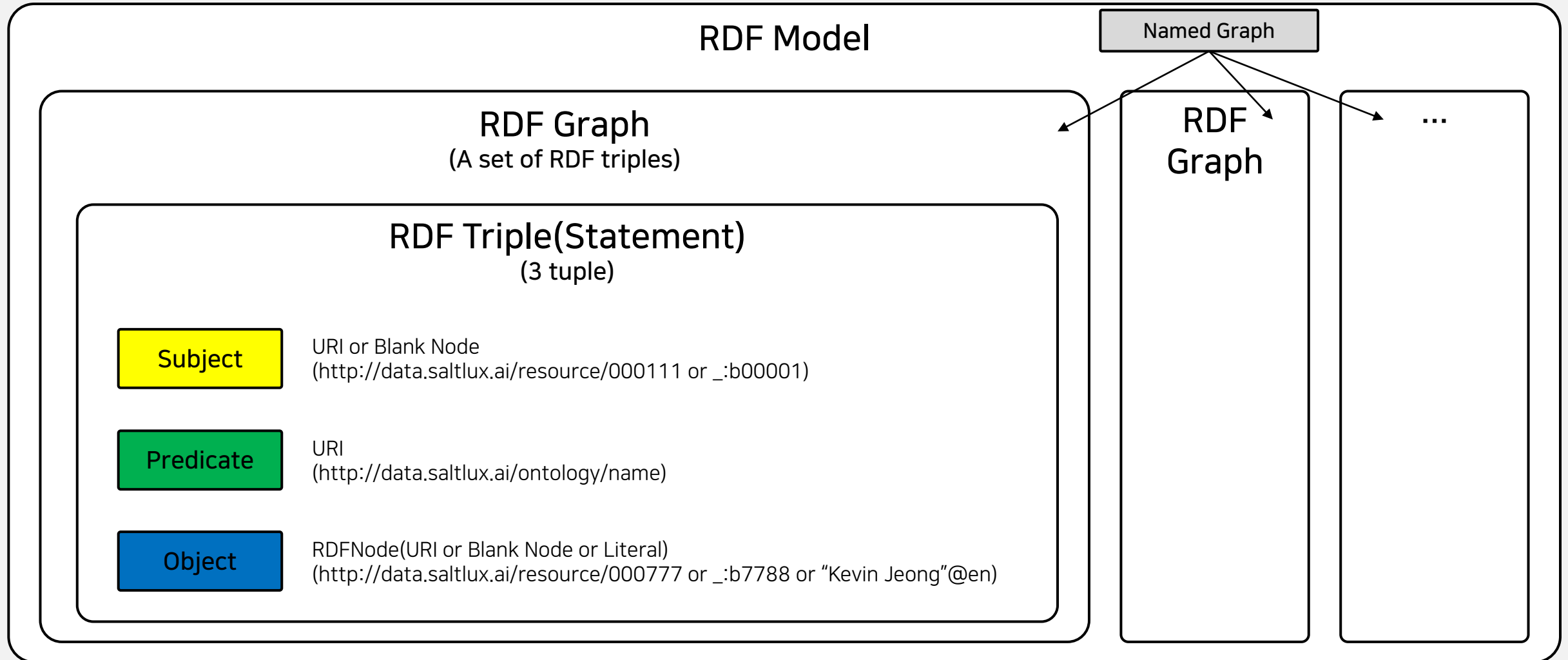
URI와 IRI

- URI: A Uniform Resource Identifier(리소스 식별자)
 - URIs are limited to a subset of the ASCII character set
- IRI: An Internationalized Resource Identifier which extends the URI.
 - IRIs may contain characters from the Universal Character Set (Unicode/ISO 10646), including Chinese or Japanese kanji, Korean, Cyrillic characters, and so forth. It is defined by RFC 3987.
 - <http://tools.ietf.org/html/rfc3987>





RDF(Resource Description Framework)





RDF Formats

- RDF는 기본적으로 RDF/XML 포맷으로 작성 할 수 있고 가독성을 높이기 위해 다양한 포맷을 제공하고 있음

RDF/XML

N-TRIPLE

TURTLE

RDF/JSON

JSON-LD

TRIG

TRIX

...

```
<?xml version="1.0"?>
```

```
<RDF>
  <Description about="http://data.saltlux.kr/resource#moonjaein">
    <rdf:type resource="http://data.saltlux.kr/ontology#Person" />
    <ont:presidentOf resource="http://data.saltlux.kr/resource#korea" />
  </Description>
  <Description about="http://data.saltlux.kr/resource#korea">
    <rdf:type resource="http://data.saltlux.kr/ontology#Country" />
  </Description>
</RDF>
```



축약구문

```
<rdf:RDF>
  <Person rdf:about="http://data.saltlux.kr/resource#moonjaein">
    <presidentOf>
      <Country rdf:about="http://data.saltlux.kr/resource#korea">
        <rdf:type rdf:resource="http://data.saltlux.kr/ontology#Description"/>
      </Country>
    </presidentOf>
    <rdf:type rdf:resource="http://data.saltlux.kr/ontology#Description"/>
  </Person>
</rdf:RDF>
```

RDF/XML

```
@prefix :      <http://data.saltlux.kr/ontology#> .
@prefix xsp:   <http://www.owl-ontologies.com/2005/08/07/xsp.owl#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf:   <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix swrlb: <http://www.w3.org/2003/11/swrlb#> .
@prefix swrl:  <http://www.w3.org/2003/11/swrl#> .
@prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .
@prefix protege:
  <http://protege.stanford.edu/plugins/owl/protege#> .
@prefix skos:  <http://www.w3.org/2004/02/skos/core#> .
@prefix rdfs:  <http://www.w3.org/2000/01/rdf-schema#> .
```

```
<http://data.saltlux.kr/resource#moonjaein>
  a          :Person , :Description ;
  :presidentOf <http://data.saltlux.kr/resource#korea> .

<http://data.saltlux.kr/resource#korea>
  a          :Country , :Description .
```

TURTLE



언어코드(@LANG)

일본어/중국어 Language Codes 예시

Country	lang	Script type
Korean	@ko	-
English	@en	-
Japanese	@ja	typically using the shortest ISO 639-1
	@ja-Hira	Hiragana
	@ja-Hrkt	Hiragana + Katakana
	@ja-Jpan	Han + Hiragana + Katakana
	@ja-Hani	Kanji (Hanzi, Hanja, Han)
	@ja-Kana	Katakana
Chinese	@zh-Hant	Traditional Chinese
	@zh-Hans	Simplified Chinese

※ 언어코드는 일반적인 언어명의 줄임(2 characters) 또는 언어와 스크립트를 사용하며, 언어와 스크립트는 하이픈(-)을 포함한다.

※ 언어코드는 "@"로 시작하며 항상 소문자이다.

IANA Language Subtag Registry:

<http://www.iana.org/assignments/language-subtag-registry>

ISO 639-1 Language Codes:

https://www.w3schools.com/tags/ref_language_codes.asp

ISO Country Codes 예시

Country	ISO Code
AFGHANISTAN	AF
ALBANIA	AL
ALGERIA	DZ
AMERICAN SAMOA	AS
ANDORRA	AD
ANGOLA	AO
JAPAN	JP
JORDAN	JO
KAZAKHSTAN	KZ
KOREA, DEMOCRATIC PEOPLE'S REPUBLIC OF	KP
KOREA, REPUBLIC OF	KR
KUWAIT	KW
KYRGYZSTAN	KG

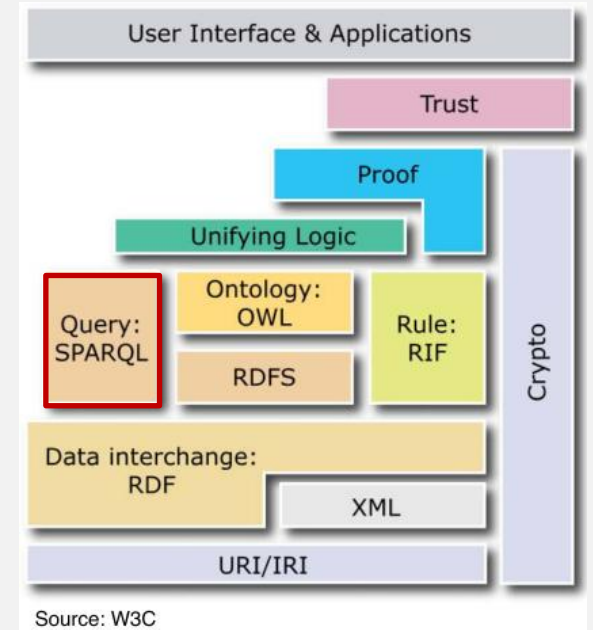


SPARQL(1/2)

- "sparkle", 스파클, SPARQL Protocol and RDF Query Language
- Turtle 형식, 그래프 패턴을 매칭함.
- RDF형식으로 저장된 데이터를 검색, 관리할 수 있음.

Query forms

- **SELECT**
 - returns all, or a subset of the variables bound in a query pattern match
 - formats : RDF/XML, JSON, CSV, TSV
- **CONSTRUCT**
 - returns an RDF graph constructed by substituting variables in a set of triple templates
- **DESCRIBE**
 - returns an RDF graph that describes the resources found.
- **ASK**
 - returns whether a query pattern matches or not.
- **SPARQL Update**
 - a standard language for specifying and executing updates to RDF graphs in a Graph Store.





SPARQL(2/2)

SPARQL 질의 작성은 다음과 같은 구성요소를 선택적으로 사용하여야 함

Prefix

PREFIX ont: <http://data.saltlux.ai/ontology/>

Query result
clause

SELECT ...

Query pattern
(Turtle형식)

WHERE {
...
}

Query modifiers

GROUP BY, HAVING, ORDER BY, LIMIT, OFFSET, VALUES, BINDINGS



- 그래프저장소 내의 데이터에 대한 업데이트를 수행함.
 - RDF그래프를 추가, 삭제할 수 있음.
- SPARQL 1.1 참조
 - INSERT DATA { ... } : RDF Statement을 추가할 경우
 - DELETE DATA { ... } : RDF Statement을 삭제할 경우
 - DELETE { ... } INSERT { ... } WHERE { ... } : 삭제와 추가를 동시에 할 경우, WHERE절의 Graph패턴에 일치할 경우 삭제/추가됨
 - INSERT INTO G { ... }, DELETE INTO G { ... } : Named graph에 트리플을 추가/삭제 할 경우
 - ※ **SPARQL Endpoint**를 통한 **UPDATE**는 트랜잭션을 지원하지 않음.
- SPARQL 1.2 참조
 - RDF-star/SPARQL-star 지원 : quoted triple 에 메타 데이터 추가, 삭제 가능 (INSERT DATA { <<s :p :o>> :source :doc1 })
 - JSON/CSV 포맷 확장 지원 : quoted triple 결과 반환



온톨로지 요구사항 명세(ORSD)와 적격질의(CQ)

Requirements Specification

Ontology Requirements Specification Document	
Domain:	Skill Management and Human Ressource Development at Company ABC
Date:	2000/08/22
Ontology Engineer:	T. Model
Goal: • The ontology serves as a guideline for the knowledge distribution between HR department and project managers • Ontology serves as a base for semantic search for employee skills	
Domain and Scope: The ontology contains concepts in the domain of human ressource development. Skills in technologies and market domains of ABC have to be modelled in detail, the common business administration area has to be modelled on a more general level. Similar domain models contain about 400 – 600 concepts.	
Supported Applications: Intranet based Skill Management System	
Knowledge Sources: • HR department web pages • Handbook about employee development, internal document • Technology and product road map, internal document • Interviews with HR department and project managers	

Competency Question

Competency Questionnaire Nr. A			
Domain:		Skill Management and Human Ressource Development at Company ABC	
Date:		2000/08/22	
Ontology Engineer:		T. Model	
CQ Nr.	Competency Question	Concepts	Relations
CQ1	Which of our consultants has experience with JAVA programming language?	Consultant <i>is a</i> employee, JAVA <i>is a</i> programming language, programming language <i>is a</i> experience, experience <i>is a</i> skill	Consultant <i>has</i> experience
CQ2	Which of our programmers worked in a project with a customer from the chemical industry?	Programmer <i>is a</i> employee, Project, Customer, Chemical industry <i>is a</i> industry, industry knowhow <i>is a</i> experience	Programmer <i>works in</i> project, project <i>at a</i> customer, Programmer <i>has</i> experience
CQ3	Is there a document about our employee	Document <i>is a</i> knowledge source,	Document <i>contains</i> knowledge, employee

온톨로지 부트캠프 - 2일차

Ontology Development (Ontology Workflow)

정 용 일 | kevin.jeong@saltlux.com

1. Knowledge Studio

온톨로지 기반 데이터 통합, 조회

Knowledge Studio (SOR v6) 기능 구성도

Semantic Ontology Repository 기반 지식그래프 플랫폼



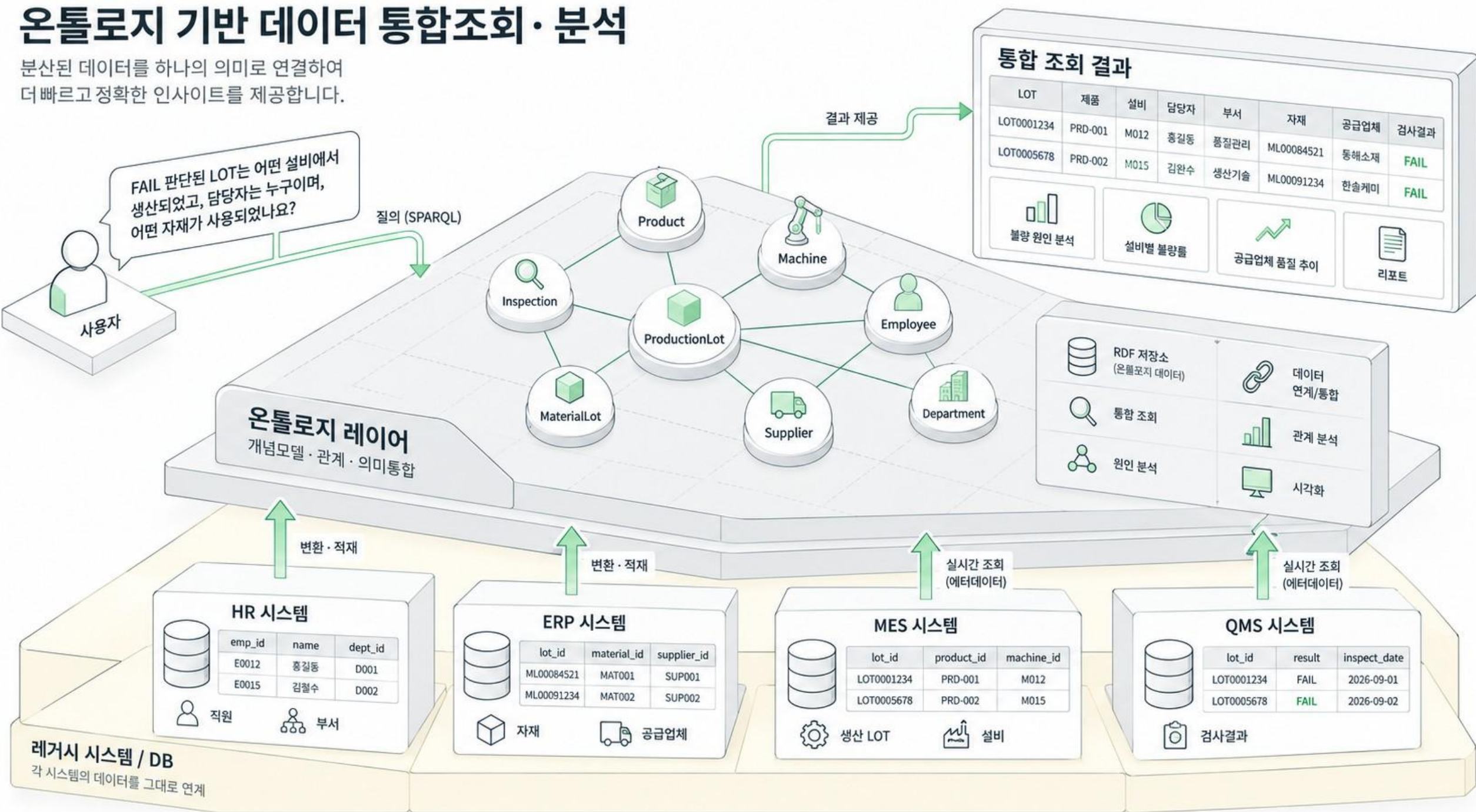


2. Ontology Workflow

실습

온톨로지 기반 데이터 통합조회·분석

분산된 데이터를 하나의 의미로 연결하여
더 빠르고 정확한 인사이트를 제공합니다.



온톨로지 워크플로우

단계	실습 내용	결과물
1. 시나리오	제조 품질 이상 추적 문제 정의	Scenario
2. CQ	업무 질문 정의	CQ 목록
3. 스키마 설계	Concept / Relationship / Property 설계	Ontology
4. 데이터매핑 / Federation	HR은 Mapping, ERP/MES/QMS는 Federation	Mapping JSON
5. 변환	Employee/Department RDF 변환	RDF Data
6. 승인본 로딩	승인 Ontology + 변환데이터 적재	SOR Repository
7. 검증	CQ 기반 SPARQL 실행	CQ 검증 결과



제조 품질 이상 원인 추적을 위한 Ontology 구축 시나리오

전자부품 제조기업 K-Smart Components는 ERP, MES, QMS, HR 시스템을 각각 운영하고 있습니다.

생산 중 품질 불량 발생하면 품질 담당자는 다음 정보를 확인해야 합니다.

어떤 생산 LOT에서 불량이 발생했는가
해당 LOT는 어떤 설비에서 생산되었는가
어떤 직원이 해당 작업을 담당했는가
담당 직원은 어느 부서 소속인가
어떤 자재 LOT가 투입되었는가
해당 자재는 어떤 공급업체에서 공급되었는가

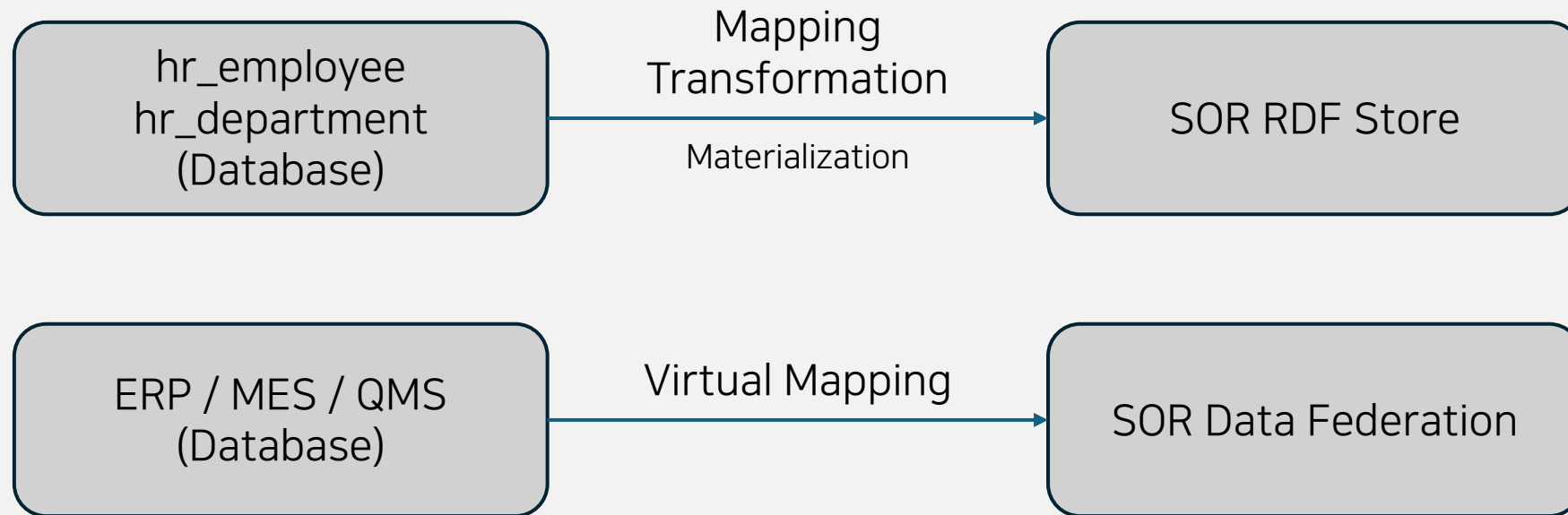
하지만 현재 데이터가 시스템별 DB 테이블에 분산되어 있어 원인 추적을 위해 여러 시스템을 개별 조회해야 합니다.

이를 해결하기 위해 제조 온톨로지를 구축하고, 일부 데이터는 RDF로 직접 적재하고 나머지는 Data Federation으로 실시간 연결합니다.



구축방향 설정

ERP, MES, QMS 데이터는 지속적으로 변경되며 데이터 규모도 큼





2. CQ(적격질의, 역량질문)

CQ1: 생산 LOT는 어떤 작업지시와 제품에 따라 어떤 설비에서 생산되었으며 담당 직원과 소속 부서는 누구이고 작업 상태는 무엇인가?

개념	관계/속성	설명
ProductionLot WorkOrder Product Machine Employee Department	ProductionLot → hasProductionLotId → http://www.w3.org/2001/XMLSchema#string ProductionLot → generatedFromWorkOrder → WorkOrder ProductionLot → manufacturedOnMachine → Machine WorkOrder → hasWorkOrderId → http://www.w3.org/2001/XMLSchema#string WorkOrder → producesProduct → Product WorkOrder → operatedByEmployee → Employee WorkOrder → hasWorkOrderStatus → http://www.w3.org/2001/XMLSchema#string Product → hasProductId → http://www.w3.org/2001/XMLSchema#string Product → hasProductName → http://www.w3.org/2001/XMLSchema#string Machine → hasMachineId → http://www.w3.org/2001/XMLSchema#string Machine → hasMachineName → http://www.w3.org/2001/XMLSchema#string Employee → hasEmployeeId → http://www.w3.org/2001/XMLSchema#string Employee → hasEmployeeName → http://www.w3.org/2001/XMLSchema#string Employee → belongsToDepartment → Department Department → hasDepartmentName → http://www.w3.org/2001/XMLSchema#string	생산 LOT를 작업지시와 제품 및 생산설비에 연결하고 작업지시의 담당 직원과 소속 부서 및 상태를 함께 조회하기 위한 질문이다.



2. CQ(적격질의, 역량질문)

CQ2: 직원은 어느 부서에 소속되어 있는가?

개념	관계/속성	설명
Employee Department	Department → hasDepartmentMember → Employee	특정 직원을 구성원으로 보유한 부서를 역방향으로 조회하여 직원의 조직 소속을 확인하기 위한 질문이다.



2. CQ(적격질의, 역량질문)

CQ3: 생산 LOT에 어떤 자재 LOT가 투입되었으며 해당 자재의 식별자와 이름 및 공급업체는 누구인가?

개념	관계/속성	설명
ProductionLot MaterialConsumption MaterialLot Material Supplier	MaterialConsumption → hasConsumptionId → http://www.w3.org/2001/XMLSchema#string MaterialConsumption → consumedForProductionLot → ProductionLot MaterialConsumption → consumesMaterialLot → MaterialLot MaterialLot → hasMaterialLotId → http://www.w3.org/2001/XMLSchema#string MaterialLot → identifiesMaterial → Material MaterialLot → suppliedBySupplier → Supplier Material → hasMaterialId → http://www.w3.org/2001/XMLSchema#string Material → hasMaterialName → http://www.w3.org/2001/XMLSchema#string Supplier → hasSupplierId → http://www.w3.org/2001/XMLSchema#string Supplier → hasSupplierName → http://www.w3.org/2001/XMLSchema#string	생산 LOT별 자재 소비 레코드를 통해 투입된 자재 LOT를 찾고 그 LOT가 나타내는 자재와 공급업체 정보를 조회하기 위한 질문이다.



2. CQ(적격질의, 역량질문)

CQ4: FAIL 생산 LOT의 생산설비와 담당 직원 및 부서 그리고 투입 자재 LOT와 자재 및 공급업체를 한 번에 추적할 수 있는가?

개념	관계/속성	설명
ProductionLot Inspection Machine Employee Department MaterialConsumption MaterialLot Material Supplier	ProductionLot → linkedFailureInspection → Inspection ProductionLot → tracedToMachine → Machine ProductionLot → tracedToOperator → Employee Employee → tracedToDepartment → Department ProductionLot → tracedThroughConsumption → MaterialConsumption MaterialConsumption → tracedToMaterialLot → MaterialLot MaterialLot → tracedToMaterial → Material MaterialLot → tracedToSupplier → Supplier	FAIL 검사와 연결된 생산 LOT를 시작점으로 설비와 작업 담당 조직 및 자재 소비 이력을 거쳐 자재 LOT와 자재 및 공급업체까지 통합 추적하기 위한 질문이다.



2. CQ(적격질의, 역량질문)

CQ5: 어떤 생산 LOT가 품질검사에서 FAIL 판정을 받았으며 검사 식별자와 검사 시각은 무엇인가?

개념	관계/속성	설명
Inspection ProductionLot	Inspection → hasInspectionId → http://www.w3.org/2001/XMLSchema#string Inspection → evaluatesProductionLot → ProductionLot Inspection → inspectedAt → http://www.w3.org/2001/XMLSchema#dateTime Inspection → hasInspectionResult → http://www.w3.org/2001/XMLSchema#string	품질검사 결과가 FAIL인 검사 레코드를 선별하고 해당 생산 LOT와 검사 식별자 및 검사 시각을 확인하기 위한 질문이다.



3. 스키마 설계

1) 온톨로지 설계는 승인된 CQ을 활용하여 자동으로 생성

2) 온톨로지 작성도우미를 통해서, 모든 엔티티에 대한 레이블, 설명을 생성

작성 모드	AI 제공자	빈 언어에 적용할 기본 언어
엔티티 도우미	openai · gpt-5.6	ko

클래스 도우미를 통해서 클래스의 상하위 관계를 생성(표준어휘 자동생성)

작성 모드	AI 제공자
클래스 도우미	openai · gpt-5.6



3. 스키마 설계

3) 설계된 온톨로지 시각화를 통해서 연결관계 확인

제조온톨로지 schema

● 작성 중 · v1 · admin · 2026-09-27T06:20

편집

새 버전

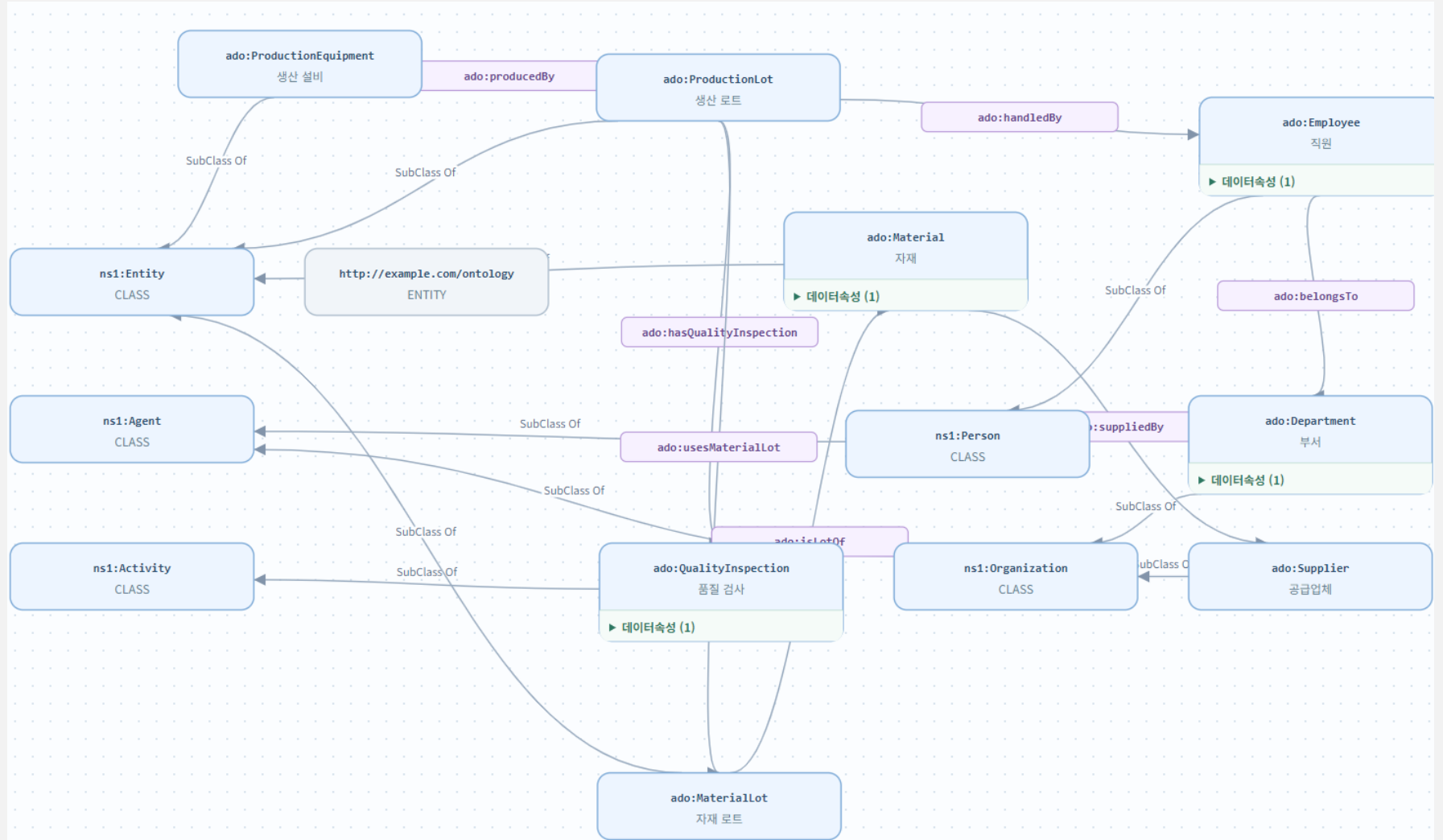
검증

검수 요청

RDF 미리보기 · 그래프

연결 · 영향 조회

-검수요청, 승인 후, 다음 단계로 넘어 갑니다.





4. 데이터 매핑

4) DB → RDF 변환규칙 설계

- 산출물 이름: [hr_department](#) 매핑
- 사용할 승인 산출물·버전 선택 · 스키마 설계: [사용할 승인된 스키마 버전 선택](#)
- SELECT / WITH SQL -> SQL 실행(샘플링)
[SELECT * FROM cloud.hr_department](#)

department_id	name
D01	생산기술팀
D02	품질관리팀
D03	구매팀

- 대상 클래스 URI
<http://example.com/ontology/Department>
- 주어 URI 규칙 (예: <https://example.org/person/{ID}>)
http://example.com/resource/{department_id}



4. 데이터 매핑

4) DB → RDF 변환규칙 설계

- 속성·규칙: 해당 클래스가 가지는 관계/데이터속성을 자동으로 불러옴. 필요시 추가 정의할 수 있음

hasDepartmentName

규칙 ID

hasDepartmentName

열 / SQL 별칭

name

속성 URI

http://example.com/ontology/hasDepartmentName

검색

레인지: http://www.w3.org/2001/XMLSchema#string

값 종류

LITERAL

객체 URI 규칙 ({열이름})

데이터타입 URI

http://www.w3.org/2001/XMLSchema#string

검색

언어 태그

변환 함수

TRIM

NULL 처리

SKIP

기본값

×



4. 데이터 매핑

4) DB → RDF 변환규칙 설계

- 산출물 이름: [hr_employee 매핑](#)
- 사용할 승인 산출물·버전 선택 · 스키마 설계: [사용할 승인된 스키마 버전 선택](#)
- SELECT / WITH SQL -> SQL 실행(샘플링)
[SELECT * FROM cloud.hr_employee](#)

employee_id	name	email	department_id
E0001	직원0001	emp0001@ksmart.local	D01
E0002	직원0002	emp0002@ksmart.local	D02
E0003	직원0003	emp0003@ksmart.local	D03

- 대상 클래스 URI
<http://example.com/ontology/Employee>
- 주어 URI 규칙 (예: <https://example.org/person/{ID}>)
http://example.com/resource/{employee_id}



4. 데이터 매핑

5) 일괄 데이터 변환

- 승인된 변환규칙에 대해서 일괄변환 진행(승인된 버전 선택)

매핑 산출물	승인 버전
hr_department 매핑	v1 · 2026-09-27T07:10:39.863295Z
hr_employee 매핑	v1 · 2026-09-27T07:10:48.521104Z

결과 출력	원본 행 범위	최대 원본 행 수	파일당 저장 트리플 수	SQL 제한 시간 (초, 0은 무제한)
파일저장	전체 행	100	100000	600

출력용 저장소 ID

manufacturing

개별 속성 규칙마다 datasources/저장소ID/ontology-workflow/규칙ID/에 분할 저장합니다. rdf:type 문장은 __rdf_type__ 폴더에 저장합니다. 저장소를 생성하거나 RDF를 적재하는 작업은



5. 변환결과

- 승인된 변환규칙에 대해서 일괄변환 진행(승인된 버전 선택)

▼ 데이터 변환 진행 상황

상태·단계·작업 ID 검색

검색

페이지 크기

5



매핑 산출물·승인 버전	상태·단계	진행	처리 행	RDF 문장	오류	경과 (초)	행/초	결과
hr_employee 매핑 v2	완료	100%	200	800 생성에 사용된 원본 행: 200 미사용 행: 0	0	0.2	893	조회
hr_department 매핑 v2	완료	100%	10	20 생성에 사용된 원본 행: 10 미사용 행: 0	0	0.1	71	조회

이전

1 / 1 · 검색 결과 2 / 전체 2

다음



6. 승인본 로딩

- 적재 저장소 선택 / 적재 저장소 생성(이미 저장소가 생성되어 있는 경우는 스키마 버전을 선택하여 반영) → 변환결과, 매핑승인본 선택 → 전체 변환결과 선택 → 위 매핑 승인본의 변환결과 로딩

대상 저장소 manufacturing

함께 적재할 승인 스키마: 제조온톨로지 schema · v1

▶ 역량 질문 (CQ) (5)



▶ 스키마 설계 (1)



▶ 데이터 매핑 (2)



▶ 변환 결과 (2)

변환 산출물·버전 검색

검색

페이지 크기

5



변환 산출물	생성 버전	트리플 수	표본 제한
hr_employee 매핑 transformation	v1 7284d5fc-e80e-4b3c-9184-7209710710d6	600	아니요
hr_department 매핑 transformation	v1 451963e7-908b-4f05-b32f-8b2f55281435	20	아니요

이전

1 / 1 · 검색 결과 2 / 전체 2

다음

위 매핑 승인본의 변환결과 로딩

프로젝트 종료



6. 승인본 로딩

• 적재 저장소 생성

적재 저장소 생성

저장소 ID

manufacturing

표시 이름

manufacturing

저장소 유형

massive-local

데이터 디렉터리

/data2/sorv6/storm-sor-6.0.0-windows-x86_64/repositories/manufacturing

SOR 설치 경로 아래 repositories/<repository-id>로 서버가 자동 생성합니다.

Massive Local 볼륨

/data2/sorv6/storm-sor-6.0.0-windows-x86_64/repositories/manufacturing/storage/volumes/volume

삭제

+ Volume 추가

SSTable을 분산 저장할 서버의 로컬 디스크 디렉터리입니다. 절대 경로를 권장하며, 비워 두면 repositories/<repository-id>/storage/volumes/volume-1을 사용합니다.

SWRL 규칙

사용 안 함

OWL/RDFS 추론과 별도로 설정합니다. 저장소를 연 뒤 SWRL 규칙 메뉴에서 규칙을 작성하고 묶음을 활성화하세요. 기존 저장소는 닫은 상태에서 설정을 저장한 후 다시 엽니다. 사용 안 함으로 변경하면 다시 열 때 SWRL 추론 결과를 제거하고 OWL 추론을 재구성합니다.

추론 프로파일

RDFS_PLUS_OPTIMIZED

확장 RDFS 규칙을 대용량 저장소에 맞게 최적화합니다.

추론 모드 (reasoning.mode / sor:reasoningMode)

HYBRID

규칙별 실행 정책을 따릅니다. 대용량 운영의 기본값이며 materialization과 필요한 query-time 규칙을 함께 사용합니다.

기본 IRI (sor:base-URL)

http://example.com/ontology

업로드한 스키마의 상대 IRI를 해석할 절대 기본 IRI입니다.

승인 스키마 등록

스키마 등록 방법

승인된 스키마 선택

승인된 스키마

제조온톨로지 schema · v2

기본 네임스페이스

http://example.com/ontology/

승인본을 Turtle 파일로 변환하거나 선택한 파일을 임시 업로드한 후 conf/<repository-id>에 등록합니다.

☐ Provenance 사용

추론된 트리플이 어떤 원본 사실과 규칙에서 도출되었는지 근거를 기록합니다. 설명 및 진실 유지 기능에 필요하지만 저장공간 사용량과 로딩·추론 비용이 증가하므로, 근거 추적이 필요하지 않으면 끄는 것을 권장합니다.

☐ 인증 없이 SPARQL 조회 허용

이 저장소의 /저장소ID/sparql에서 SELECT, ASK, CONSTRUCT, DESCRIBE 조회를 허용합니다. 데이터 변경과 관리 API에는 인증이 필요합니다.

☐ 인증 없이 Cypher 조회 허용

이 저장소의 /저장소ID/cypher POST 조회만 허용합니다. 설정·통계·종지 API는 인증이 필요하며 쓰기 구문은 지원하지 않습니다.



7. 검증

- CQ별 검증 쿼리 등록 및 실행을 통해 원하는 결과가 나오는 지 결과 확인

CQ 질문	작업자	상태	SPARQL	관리
생산 LOT는 어떤 작업지시와 제품에 따라 어떤 설비에서 생산되었으며 담당 직원과 소속 부서는 누구이고 작업 상태는 무엇인가? · v1 576dd0d0-773b-4c64-b88b-29bfe59cb2e9	admin	APPROVED	▶ 쿼리 조회	쿼리 등록·편집 실행
직원은 어느 부서에 소속되어 있는가? · v1 597aab9b-92f7-41aa-9d56-e2fd3095dab8	admin	APPROVED	▶ 쿼리 조회	쿼리 등록·편집 실행
어떤 생산 LOT가 품질검사에서 FAIL 판정을 받았으며 검사 식별자와 검사 시각은 무엇인가? · v1 83afd1c8-bcd2-434f-a1d4-02a728a0f7a7	admin	APPROVED	▶ 쿼리 조회	쿼리 등록·편집 실행
FAIL 생산 LOT의 생산설비와 담당 직원 및 부서 그리고 투입 자재 LOT와 자재 및 공급업체를 한 번에 추적할 수 있는가? · v1 7d312f0f-5e37-46c8-8965-5fa24e795ea2	admin	APPROVED	▶ 쿼리 조회	쿼리 등록·편집 실행
생산 LOT에 어떤 자재 LOT가 투입되었으며 해당 자재의 식별자와 이름 및 공급업체는 누구인가? · v1 66664ad4-7cc1-4127-a7b0-0c08d5905bfb	admin	APPROVED	▶ 쿼리 조회	쿼리 등록·편집 실행

http://211.109.9.83:6999/admin/main/boot/sorv6_validation_5queries.sparql



3. SPARQL 실습



1. Data federation

SPARQL

- 레거시DB와 실시간 연계를 위한 Data federation

▼ 활성 Virtual Graph 9개 집기

매핑 ID · Graph IRI 검색

검색어를 입력하세요 9 / 9개

erp_supplier v2

urn:virtual:erp:supplier

기본 그래프에도 포함

erp_material v2

urn:virtual:erp:material

기본 그래프에도 포함

erp_product v2

urn:virtual:erp:product

기본 그래프에도 포함

erp_material_lot v2

urn:virtual:erp:material-lot

기본 그래프에도 포함

mes_machine v2

urn:virtual:mes:machine

기본 그래프에도 포함

mes_work_order v2

urn:virtual:mes:work-order

기본 그래프에도 포함

mes_production_lot v2

urn:virtual:mes:production-lot

기본 그래프에도 포함

mes_material_consumption v2

urn:virtual:mes:material-consumption

기본 그래프에도 포함

qms_inspection v2

urn:virtual:qms:inspection

기본 그래프에도 포함

01 외부 데이터소스

manufacturing ▼

소스 ID

manufacturing

DB 종류

MYSQL ▼

JDBC URL

jdbc:mysql://211.109.9.74:3306/cloud

JDBC Driver Class (선택)

org.gjt.mm.mysql.Driver

읽기 전용 DB 계정

cloud

비밀번호 환경변수 이름

MANUFACTURING_DB_PASSWORD

Catalog (선택)

Schema (선택)

접근 사용자 (선택 구분)

02 Virtual Mapping

mes_material_consumption ▼

매핑 ID

mes_material_consumption

새 버전 번호

3

데이터소스

manufacturing ▼

Virtual Graph IRI

urn:virtual:mes:material-consumption

클래스 IRI

http://example.com/ontology/MaterialConsumption

주어 IRI 템플릿

http://example.com/resource/material-consumption/{consumption_id}

SQL 샘플 조회 후 첫 번째 행으로 IRI를 표시합니다.

Schema (선택)

Table / View (SQL 입력 시 비워 두세요)

논리 테이블 SQL (SELECT / WITH)

SQL

자동완성

1 SELECT consumption_id, production_lot_id, material_lot_id FROM mes_material_consumption



1. 질의실습

SPARQL

- 직원이 속한 부서, productionLot_workorder_machine

```
SELECT ?employeeId ?employeeName ?departmentName
WHERE {
  ?employee a ado:Employee ;
    ado:hasEmployeeId ?employeeId ;
    ado:hasEmployeeName ?employeeName ;
    ado:belongsToDepartment ?department .

  ?department a ado:Department ;
    ado:hasDepartmentName ?departmentName .
}
ORDER BY ?employeeId
LIMIT 100
```

```
SELECT ?lotId ?workOrderId ?machineId ?machineName
WHERE {
  ?lot a ado:ProductionLot ;
    ado:hasProductionLotId ?lotId ;
    ado:generatedFromWorkOrder ?workOrder ;
    ado:manufacturedOnMachine ?machine .

  ?workOrder a ado:WorkOrder ;
    ado:hasWorkOrderId ?workOrderId .

  ?machine a ado:Machine ;
    ado:hasMachineId ?machineId ;
    ado:hasMachineName ?machineName .
}
ORDER BY ?lotId
LIMIT 100
```



1. 질의실습

SPARQL

- workorder_product_employee_department

```
SELECT ?workOrderId ?productId ?productName ?employeeId ?employeeName ?departmentName
WHERE {
  ?workOrder a ado:WorkOrder ;
    ado:hasWorkOrderId ?workOrderId ;
    ado:producesProduct ?product ;
    ado:operatedByEmployee ?employee .

  ?product a ado:Product ;
    ado:hasProductId ?productId ;
    ado:hasProductName ?productName .

  ?employee a ado:Employee ;
    ado:hasEmployeeId ?employeeId ;
    ado:hasEmployeeName ?employeeName ;
    ado:belongsToDepartment ?department .

  ?department a ado:Department ;
    ado:hasDepartmentName ?departmentName .
}
ORDER BY ?workOrderId
LIMIT 100
```



1. 질의실습

SPARQL

- workorder_product_employee_department

```
PREFIX ado: <http://example.com/ontology/>
```

```
# CQ3. 생산 LOT에 어떤 자재 LOT가 투입되었고
```

```
# 해당 자재와 공급업체는 누구인가?
```

```
SELECT ?lotId
```

```
    ?materialLotId
```

```
    ?materialId ?materialName
```

```
    ?supplierId ?supplierName
```

```
WHERE {
```

```
    ?consumption a ado:MaterialConsumption ;
```

```
        ado:consumedForProductionLot ?lot ;
```

```
        ado:consumesMaterialLot ?materialLot .
```

```
    ?lot a ado:ProductionLot ;
```

```
        ado:hasProductionLotId ?lotId .
```

```
    ?materialLot a ado:MaterialLot ;
```

```
        ado:hasMaterialLotId ?materialLotId ;
```

```
        ado:identifiesMaterial ?material ;
```

```
        ado:suppliedBySupplier ?supplier .
```

```
    ?material a ado:Material ;
```

```
        ado:hasMaterialId ?materialId ;
```

```
        ado:hasMaterialName ?materialName .
```

```
    ?supplier a ado:Supplier ;
```

```
        ado:hasSupplierId ?supplierId ;
```

```
        ado:hasSupplierName ?supplierName .
```

```
}
```

```
ORDER BY ?lotId ?materialLotId
```

```
LIMIT 100
```



1. 질의실습

CYPHER

http://211.109.9.83:6999/admin/main/boot/sorv6_validation_queries_supported.cypher

2026소버린온톨로지부트캠프 교육

THANK YOU

